

PROJECT REPORT

4×4 Matrix Multiplication Using 8086 Assembly Language

Course: Computer Architecture

Project: Implement matrix (4×4) multiplication using
8086 assembly language

1. Objective

The main objective of this project is to implement 4×4 matrix multiplication using assembly language. The program stores two predefined 4×4 matrices, performs the multiplication using three nested loops, stores the resulting values in a third matrix, and displays the result as a 4×4 matrix in the terminal.

The project also demonstrates register-based indexing, arithmetic operations, memory addressing, loops, and Linux system calls for character output and program termination.

2. Theory

2.1 Matrix Multiplication

For two 4×4 matrices A and B, the product matrix C is calculated as:

$$C[i][j] = \sum A[i][k] \times B[k][j], \text{ for } k = 0 \text{ to } 3$$

Each element of C is obtained by taking one row from matrix A and one column from matrix B, multiplying their corresponding elements, and adding the four products.

2.2 8086 Assembly Language Concepts Used

- Registers: General-purpose registers are used for loop counters, indexes, addresses, and arithmetic values.
- Memory addressing: Matrix elements are accessed using calculated linear indexes.
- Shift operation: SHL by 2 is used to calculate $i \times 4$ or $k \times 4$ because each matrix row contains four byte elements.
- IMUL instruction: Performs signed integer multiplication of the loaded matrix values.
- Loops and conditional jumps: CMP, JGE, INC, and JMP control the three nested loops.
- Linux system calls: `int 0x80` is used for writing output and exiting the program.

3. Design

3.1 Input Matrices

The program uses the following predefined matrices. Matrix B is the 4×4 identity matrix.

Matrix A

1 2 3 4

5 6 7 8

9 1 2 3
4 5 6 7

Matrix B

1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1

3.2 Algorithm

1. Initialize the row index i to 0.
2. For each row i , initialize the column index j to 0.
3. For each column j , initialize the inner index k to 0 and clear the accumulator.
4. Calculate the index $i \times 4 + k$ and load $A[i][k]$.
5. Calculate the index $k \times 4 + j$ and load $B[k][j]$.
6. Multiply $A[i][k]$ and $B[k][j]$, then add the product to the accumulator.
7. Repeat the inner loop for $k = 0$ to 3.
8. Store the accumulated value in $C[i][j]$.
9. Repeat for all four columns and four rows.
10. Print the 16 result elements in a 4×4 format.

3.3 Program Flow

The design follows a three-level nested-loop structure: the outer loop selects a row of A , the middle loop selects a column of B , and the inner loop computes the dot product for one result element.

4. Implementation

The implementation is written in NASM-style assembly and is intended to run in a Linux 32-bit environment using the `int 0x80` system-call interface. The online compiler used for the implementation/testing is Tutorialspoint Online Assembly (NASM) Compiler:

<https://www.tutorialspoint.com/compiler/online-assembly-compiler.htm>

4.1 Source Code

```
section .data
    matrixA db 1, 2, 3, 4
             db 5, 6, 7, 8
             db 9, 1, 2, 3
             db 4, 5, 6, 7

    matrixB db 1, 0, 0, 0
             db 0, 1, 0, 0
             db 0, 0, 1, 0
             db 0, 0, 0, 1

    space db ''
    newline db 10

section .bss
    matrixC resb 16
    char_buf resb 1

section .text
    global _start

_start:
    ; --- Matrix Multiplication ---
    mov ebx, 0          ; i (Row = 0..3)

outer_loop:
    cmp ebx, 4
    jge print_result

    mov edi, 0          ; j (Col = 0..3)

middle_loop:
    cmp edi, 4
    jge next_row

    mov esi, 0          ; k (Inner = 0..3)
```

```
xor edx, edx      ; Sum accumulator
```

```
inner_loop:
```

```
    cmp esi, 4  
    jge store_result
```

```
    ; Get A[i][k] -> index = i * 4 + k  
    mov eax, ebx  
    shl eax, 2  
    add eax, esi  
    movzx ecx, byte [matrixA + eax]
```

```
    ; Get B[k][j] -> index = k * 4 + j  
    mov eax, esi  
    shl eax, 2  
    add eax, edi  
    movzx eax, byte [matrixB + eax]
```

```
    ; Multiply A[i][k] * B[k][j]  
    imul eax, ecx  
    add edx, eax
```

```
    inc esi  
    jmp inner_loop
```

```
store_result:
```

```
    ; Store in Matrix C -> index = i * 4 + j  
    mov eax, ebx  
    shl eax, 2  
    add eax, edi  
    mov [matrixC + eax], dl
```

```
    inc edi  
    jmp middle_loop
```

```
next_row:
```

```
    inc ebx
```

```
jmp outer_loop
```

```
; --- Print Result Matrix ---
```

```
print_result:
```

```
    mov esi, 0
```

```
print_loop:
```

```
    cmp esi, 16
```

```
    jge done
```

```
    mov al, [matrixC + esi]
```

```
    add al, '0'
```

```
    mov [char_buf], al
```

```
; Print Character (Linux syscall)
```

```
    mov eax, 4
```

```
    mov ebx, 1
```

```
    mov ecx, char_buf
```

```
    mov edx, 1
```

```
    int 0x80
```

```
; Print Space
```

```
    mov eax, 4
```

```
    mov ebx, 1
```

```
    mov ecx, space
```

```
    mov edx, 1
```

```
    int 0x80
```

```
; Check if newline needed (every 4 elements)
```

```
    mov eax, esi
```

```
    inc eax
```

```
    xor edx, edx
```

```
    mov ebx, 4
```

```
    div ebx
```

```
    cmp edx, 0
```

```
    jne back_to_print
```

```
; Print Newline
mov eax, 4
mov ebx, 1
mov ecx, newline
mov edx, 1
int 0x80
```

```
back_to_print:
    inc esi
    jmp print_loop
```

```
done:
    ; Exit program (sys_exit)
    mov eax, 1
    xor ebx, ebx
    int 0x80
```

4.2 Code Explanation

Data section: Defines matrixA and matrixB as byte arrays. The space and newline characters are also defined for output formatting.

BSS section: Reserves 16 bytes for matrixC, which stores the 16 elements of the result, and one byte for char_buf.

Outer loop: EBX represents the row index i. It runs from 0 to 3.

Middle loop: EDI represents the column index j. It runs from 0 to 3.

Inner loop: ESI represents k. The accumulator in EDX stores the sum of four multiplication terms.

Index calculation: SHL by 2 multiplies the row/inner index by four to obtain the correct byte offset in a 4-column matrix.

Multiplication: The A and B elements are loaded with MOVZX, multiplied with IMUL, and added to EDX.

Result storage: The accumulated low byte DL is stored in matrixC at index $i \times 4 + j$.

Output: Each result byte is converted to a single ASCII digit by adding '0' and printed with the Linux sys_write system call.

5. Testing

Testing was performed by executing the program in the online NASM compiler. The test used the predefined Matrix A and the 4×4 identity Matrix B. Because multiplying a matrix by the identity matrix should preserve the original matrix, the expected result is the same as Matrix A.

Test Case	Input	Expected Output	Observed Output
TC-01	$A \times I$	Same as Matrix A	1 2 3 4 / 5 6 7 8 / 9 1 2 3 / 4 5 6 7

6. Result

The program successfully performs the 4×4 matrix multiplication for the provided matrices. Since Matrix B is the identity matrix, the mathematical result is Matrix A:

```
1 2 3 4
5 6 7 8
9 1 2 3
4 5 6 7
```

The supplied execution screenshot shows the same 4×4 matrix in the terminal output, confirming the expected result for this test case.

7. Conclusion and Future Work

This project demonstrates the implementation of 4×4 matrix multiplication using assembly language and low-level programming concepts. The program uses nested loops, register-based indexing, memory addressing, multiplication, accumulation, and Linux system calls. The output confirms that the implementation produces the expected result for the selected identity-matrix test case.

Future work may include accepting matrix values from the user, supporting matrices larger than 4×4, displaying multi-digit and negative results correctly, adding input validation, and improving the output formatting. These extensions would make the program more general and practical.

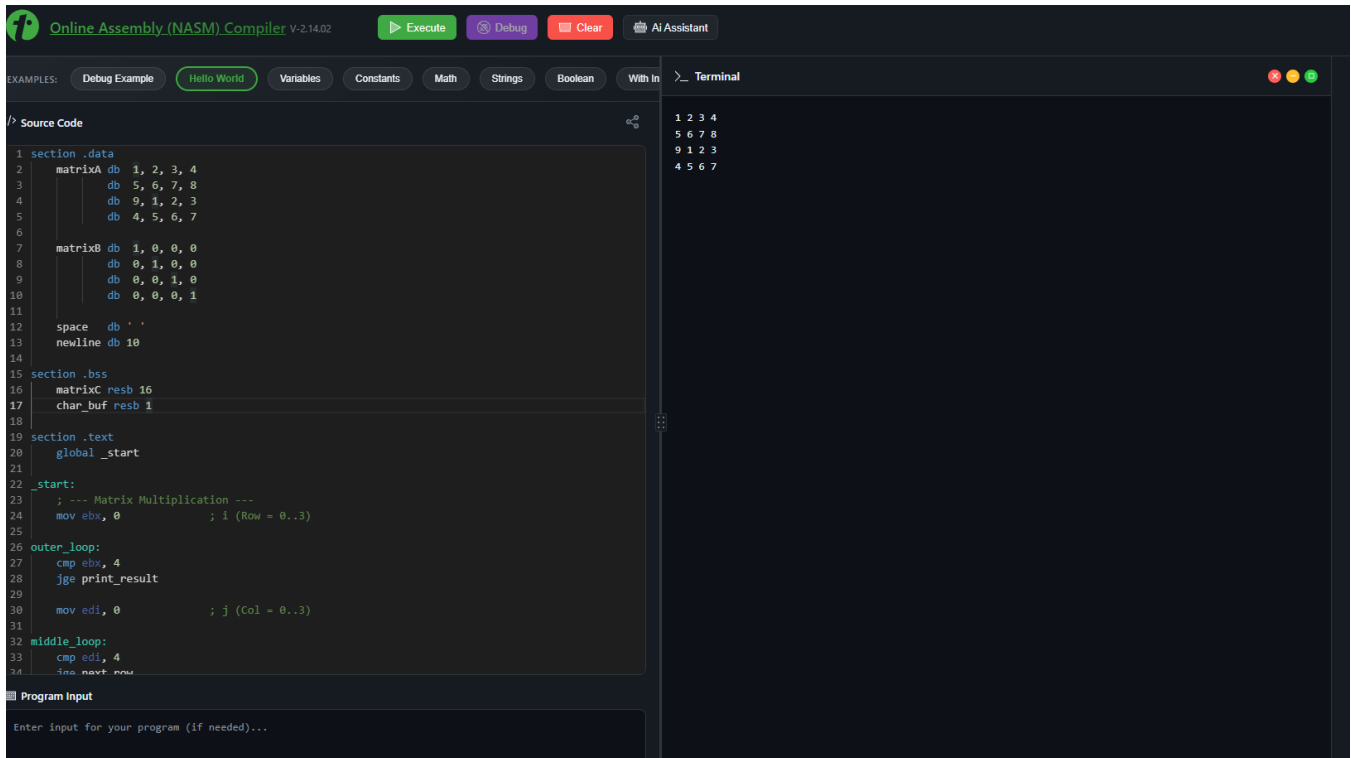
Appendix: Output

The execution output for the selected test case is:

1 2 3 4
5 6 7 8
9 1 2 3
4 5 6 7

8. Output Screenshot

The following screenshot shows the program execution in the online NASM compiler.



The screenshot displays the Online Assembly (NASM) Compiler interface. The top bar includes the logo, version (V-2.14.02), and buttons for Execute, Debug, Clear, and AI Assistant. Below the bar are tabs for EXAMPLES: Debug Example, Hello World (selected), Variables, Constants, Math, Strings, Boolean, and With In. The main area is divided into Source Code and Program Input sections. The Source Code section contains the following assembly code:

```
1 section .data
2     matrixA db 1, 2, 3, 4
3             db 5, 6, 7, 8
4             db 9, 1, 2, 3
5             db 4, 5, 6, 7
6
7     matrixB db 1, 0, 0, 0
8             db 0, 1, 0, 0
9             db 0, 0, 1, 0
10            db 0, 0, 0, 1
11
12     space db ' '
13     newline db 10
14
15 section .bss
16     matrixC resb 16
17     char_buf resb 1
18
19 section .text
20     global _start
21
22 _start:
23     ; --- Matrix Multiplication ---
24     mov ebx, 0 ; i (Row = 0..3)
25
26 outer_loop:
27     cmp ebx, 4
28     jge print_result
29
30     mov edi, 0 ; j (Col = 0..3)
31
32 middle_loop:
33     cmp edi, 4
34     jne next_row
```

The Program Input section contains the text: Enter input for your program (if needed)...

The Terminal section on the right shows the output of the program execution:

```
1 2 3 4
5 6 7 8
9 1 2 3
4 5 6 7
```

Figure 1: Program execution and 4×4 matrix output